BDA : Module 2

Q.1) Secondary nameNode is backup of namenode: True or false ?

⇒ a) Secondary nameNode is not a backup of nameNode.

b) Its main job is to periodically merge Namenode's edit logs with File system Image.

c) It doesn't recieve real time updates from Name-Node.

d) In case Namenode crashes; secondary one can't immediately take over.

e) Manual recovery needed.

Q.2) Write map reduce pseudo-code for word count. Apply map reduce working:
"This is NoSQL. NoSQL handles complex data".

⇒ 1) Preprocessing (lower), remove punctuation)
"this is nosql nosql handles complex data".

Map function: has statement

```
function map (document_id, document)
    for each word in document.split():
        emit (word.lower(), 1)
```

input:
map (1, " ______ ,, ______ ")

output :

("this", 1)
("is", 1)
("nosql", 1)
("nosql", 1)
("handles", 1)
("complex", 1)
("data", 1

2) Shuffle & Sort [group values by word]

"this" -> [1]
"is" -> [1]
"nosql" -> [1, 1]
"handles" -> [1]
"complex" -> [1]
"data" -> [1]

3) Reduce function
 └ pass each word

```
function (word, values_list):
    sum = 0
    for value in values_list
        sum += value
    emit (word, sum)
```

O/P:

("this", 1)
("is", 1)
("nosql", 2)
("handles", 1)   (data, 1)
("complex", 1)

Q.3) Explain MapReduce pipeline with suitable example.

⟹

a) MapReduce is a programming model used for processing large scale data in distributed environment.

b) 3 key phases:

1) Map
2) Shuffle & Sort
3) Reduce

c) All these phases run in ||el across hadoop cluster.

d) Eg: Cat sat on the mat. Cat chased rat.

1) Preprocessing:

cat sat on the mat cat chased rat

split 1: cat sat on the mat

split 2: cat chased rat

Each split sent to separate mapper.

O/P

Mapper 1:	Mapper 2:
("cat", 1)	("cat", 1)
("sat", 1)	("chased", 1)
("on", 1)	("rat", 1)
("the", 1)	
("mat", 1)	

2) Shuffle & Sort phase

[group values with same key, sort them.]

O/P :

("cat", [1,1])

("sat", [1])

("on", [1])

("the", [1])

("mat", [1])

("chased", [1])

("rat", [1])

3) Reduce phase.

("cat", 2)

("sat", 1)

("on", 1)

("the", 1)

("mat", 1)

("chased", 1)

("rat", 1)

8.5) Selection Operation using MapReduce

Algorithm:

```
map (key, value):
    for tuple in value:
        if tuple satisfies condition:
            emit (tuple, tuple)

reduce (key, values)
    emit (key, key)
```

Map Worker 1

A	B
1	2
3	1

A	B
1	2
3	5

Map Worker 2

A	B
2	3
4	5

A	B
1	1
2	1

Condition : B <= 2

1. MW1

key	value
(1,2)	(1,2) (1,2)
(3,1)	(3,1)

MW2

key	value
(1,1)	(1,1)
(2,1)	(2,1)

A hash function applied:

2. MW1

key	value
(1,2)	(1,2) (1,2)

key	value
(3,1)	(3,1)

MW2

key	value
(1,1)	(1,1)

K	V
(2,1)	(2,1)

(3) RW1 | RW2

~~keep~~ exchange tables 2, 3 [to discard key redundancies]

k	v
(1,2)	(1,2)
	(1,2)

k	v
(1,1)	(1,1)

k	v
(3,1)	(3,1)

k	v
(2,1)	(2,1)

(4) RW1 (key, key) | RW2

k	v
(1,2)	(1,2) (1,2)
(1,1)	(1,1)

k	v
(3,1)	(3,1)
(2,1)	(2,1)

(5) RW1

A	B
1	2
1	1

RW2

A	B
3	1
2	1

Q. 6) Projection using MapReduce
Let S be subset containing selected attributes (column)

map(key, value):
  for tuples in value:
    ts = tuples with attributes in S.
  emit (ts, ts)

reduce (key, values):
  emit (key, key)

Qn) Project (B, C)

MW 1

A	B	C
1	2	3
3	1	4

A	B	C
2	2	3
4	3	4

MW 2

A	B	C
5	7	2
7	1	1

A	B	C
3	2	5
4	1	0

(1)

MW 1

K	V
(2, 3)	(2, 3) (2, 3)
(1, 4)	(1, 4)
(3, 4)	(3, 4)

MW 2

K	V
(7, 2)	(7, 2)
(1, 1)	(1, 1)
(2, 5)	(2, 5)
(1, 0)	(1, 0)

A hash function will be applied:

(2) MW1

k	v
(2,3)	(2,3)(2,3)
(1,4)	(1,4)

k	v
(3,4)	(3,4)

MW2

k	v
(7,2)	(7,2)
(1,1)	(1,1)

k	v
(2,5)	(2,5)
(1,0)	(1,0)

(3) RW1

k	v
(2,3)	(2,3)(2,3)
(1,4)	(1,4)

k	v
(7,2)	(7,2)
(1,1)	(1,1)

RW2

k	v
(3,4)	(3,4)

k	v
(2,5)	(2,5)
(1,0)	(1,0)

(4) RW1

k	v
(2,3)	(2,3)(2,3)
(1,4)	(1,4)
(7,2)	(7,2)
(1,1)	(1,1)

RW2

k	v
(3,4)	(3,4)
(2,5)	(2,5)
(1,0)	(1,0)

Only keys

(5) RW1

B	C
2	3
1	4
7	2
1	1

RW2

B	C
3	4
2	5
1	0

Q.7) Union using Map Reduce

⟹ map (key, value):
    for tuple in value:
        emit (tuple, tuple)

reduce (key, values):
    emit (key, key)

MW1

A	B
1	2
3	1

A	B
1	2
2	1

MW2

A	B
2	3
4	5

A	B
1	1
2	1

(1)

MW1

K	V
(1,2)	(1,2)(1,2)
(3,1)	(3,1)
(2,1)	(2,1)

MW2

K	V
(2,3)	(2,3)
(4,5)	(4,5)
(1,1)	(1,1)
(2,1)	(2,1)

apply hash function

(2)

MW1

K	V
(1,2)	(1,2)(1,2)
(3,1)	(3,1)

K	V
(2,1)	(2,1)

MW2

K	V
(2,3)	(2,3)
(4,5)	(4,5)

K	V
(1,1)	(1,1)
(2,1)	(2,1)

(3) RW1 Discard key redundancies RW2

k	v
(1,2)	(1,2)(1,2)
(3,1)	(3,1)

k	v
(2,3)	(2,3)
(4,5)	(4,5)

k	v
(2,1)	(2,1)

k	v
(1,1)	(1,1)
(2,1)	(2,1)

(4) RW1

k	v
(1,2)	(1,2)
(3,1)	(3,1)
(2,3)	(2,3)
(4,5)	(4,5)

RW2

k	v
(2,1)	(2,1)(2,1)
(1,1)	(1,1)

(5) RW1

A	B
1	2
3	1
2	3
4	5

RW2 (key, key)

A	B
2	1
1	1

Q.8) Intersection using MapReduce

```
map (key , value):
    for tuple in value:
        emit (tuple, tuple)

reduce (key, values):
    if values = [key, key]
        emit (key, key)
```

MW1

A	B
1	2
3	1

A	B
1	2
2	1

MW2:

A	B
2	3
4	5

A	B
1	1
2	1

① MW1

k	v
(1,2)	(1,2)(1,2)
(3,1)	(3,1)
(2,1)	(2,1)

MW 2

k	v
(2,3)	(2,3)
(4,5)	(4,5)
(1,1)	(1,1)
(2,1)	(2,1

apply hash function:

② MW1

k	v
(1,2)	(1,2)(1,2)
(3,1)	(3,1)

k	v
(2,1)	(2,1)

MW 2

k	v
(2,3)	(2,3)
(4,5)	(4,5)

k	v
(1,1)	(1,1)
(2,1)	(2,1)

(3) RW1 Discard key redundancies RW2

RW1

k	v
(1,2)	(1,2)(1,2)
(3,1)	(3,1)

k	v
(2,3)	(2,3)
(4,5)	(4,5)

RW2

k	v
~~[illegible]~~	
~~[illegible]~~	
(2,1)	(2,1)

k	v
(1,1)	(1,1)
(2,1)	(2,1)

(4) RW1

k	v
(1,2)	(1,2)(1,2)
(3,1)	(3,1)
(2,3)	(2,3)
(4,5)	(4,5)

RW 2

k	v
(2,1)	(2,1)(2,1)
(1,1)	(1,1)

(5) if len(values) > 1 ; then commit

RW1

k	v
(1,2)	(1,2)(1,2)

RW2

k	v
(2,1)	(2,1)(2,1)

(6) RW 1

A	B
1	2

RW 2

A	B
2	1

9) ~~map (key, val~~ Difference using MapReduce

```
map (key, value):
    if key == R:
        for tuple in value:
            emit (tuple, R)
    else:
        for tuple in value:
            emit (tuple, S)

reduce (key, values)
    if values == [R]
        emit (key, key)
```

Q^n)

MW 1

T1

A	B
1	2
3	1

T2

A	B
1	2
2	1

MW 2

T1

A	B
2	3
4	5

T2

A	B
1	1
2	1

⇒ (1)

MW 1

k	v
(1,2)	[T2,T2]
(3,1)	[T1]
(2,1)	[T2]

MW 2

k	v
(2,3)	[T1]
(4,5)	[T1]
(1,1)	[T2]
(2,1)	[T2]
	[T2]

Apply hash function:

(2)

MW 1

k	v
(1,2)	[T1,T2]
(3,1)	[T1]

k	v
(2,1)	[T2]

MW 2

k	v
(2,3)	[T1]
(4,5)	[T1]

k	v
(1,1)	[T2
(2,1)	[T

(3) Discard key redundancies

RW1

k	v
(1,2)	[T1,T2]
(3,1)	[T1]

k	v
(2,3)	[T1]
(4,5)	[T1]

RW 2

k	v
(2,1)	[T2]

k	v
(1,1)	[T2]
(2,1)	[T2]

(4) RW1

k	v
(1,2)	[T1,T2]
(3,1)	[T1]
(2,3)	[T1]
(4,5)	[T1]

RW 2

k	v
(2,1)	[T2]
(1,1)	[T2]

Both MW1 & MW2 it way in T2

Eliminate keys both in T1 & T2; only in T2

(5) RW1

k	v
~~(1,2)~~	
(3,1)	[T1]
(2,3)	[T1]
(4,5)	[T1]

RW2

k	v

(6) RW1

A	B
3	1
2	3
4	5

RW 2

A	B

Big Data Analytics.

Page No. | Date

* **Matrix Multiplication using Map Reduce.**

$A_{ij} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ $B_{jk} = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$

→ **Mapper for Matrix A**

A (K, V) = ((i, k) (A, j, A_{ij}) for all k.

Mapper for Matrix B

B (K, V) = (i, k) (B, j, B_{jk}) for all i.

* Tompute Mapper for matrix A :-

For K=1:

- K=1 ; i=1 , j=1 ⟹ (1,1) (A, 1, A_{11}) ⟹ (1,1) (A, 1, 1)
- K=1 ; i=1 ; j=2 ⟹ (1,1) (A, 2, A_{12}) ⟹ (1,1) (A, 2, 2)
- K=1 ; i=2 ; j=1 ⟹ (2,1) (A, 1, A_{21}) ⟹ (2,1) (A, 1, 3)
- ~~K=2~~ K=1 ; i=2 j=2 ⟹ (2,1) (A, 2, A_{22}) ⟹ (2,2) (A, 2, 4)

For K=2:

- K=2 ; i=1 ; j=1 ⟹ (1,2) (A, 1, A_{11}) ⟹ (1,2) (A, 1, 1)
- K=2 ; i=1 ; j=2 ⟹ (1,2) (A, 2, A_{12}) ⟹ (1,2) (A, 2, 2)
- K=2 ; i=2 ; j=1 ⟹ (2,2) (A, 2, A_{21}) ⟹ (2,1) (A, 2, 3)
- K=2 ; i=2 ; j=2 ⟹ (2,2) (A, 2, A_{22}) ⟹ (2,2) (A, 2, 4)

* Tompute Mapper for Matrix B :-

For K=1:

- K=1 ; i=1 ; j=1 ⟹ (1,1) (B, 1, B_{11}) ⟹ (1,1) (B, 1, 5)
- K=1 ; i=1 ; j=2 ⟹ (1,1) (B, 2, B_{21}) ⟹ (1,1) (B, 2, 7)
- K=1 ; i=2 ; j=1 ⟹ (2,1) (B, 1, B_{11}) ⟹ (2,1) (B, 1, 5)
- K=1 ; i=2 ; j=2 ⟹ (2,1) (B, 2, B_{21}) ⟹ (2,1) (B, 2, 7)

For K=2:

- K=2 ; i=1 ; j=1 ⟹ (1,2) (B, 1, B_{12}) ⟹ (1,2) (B, 1, 6)
- K=2 ; i=1 ; j=2 ⟹ (1,2) (B, 2, B_{22}) ⟹ (1,2) (B, 2, 8)
- K=2 ; i=2 ; j=1 ⟹ (2,2) (B, 1, B_{12}) ⟹ (2,2) (B, 1, 6)
- K=2 ; i=2 ; j=2 ⟹ (2,2) (B, 2, B_{22}) ⟹ (2,2) (B, 2, 8)

Standard
Fuzzy c
Eg:- x=
fuzz
: fuzz
Fuzzy
Fuzzy

Page No.
Date

* <u>Computing the Reducer</u>

Common pairs :- (1,1), (1,2), (2,1), (2,2) — Multiply list values

(1,1), Alist = { (A, 1, 1), (A, 2, 2) }
Blist = { (B, 1, 5), (B, 2, 7) }

Now $A_{ij} \times B_{jk} = \{ (1 \times 5) + (2 \times 7)] = \underline{\underline{19}}$

Similarly, Calculate for {(1,2), (2,1), (2,2)}

(1,2) ; Alist = { (A, 1, 1), (A, 2, 2) }
Blist = { (B, 1, 6), (B, 2, 8) }
Now $A_{ij} \times B_{jk} = 1 \times 6 + 2 \times 8 = 6 + 16 = \underline{\underline{22}}$

(2,1) ; Alist = { (A, 1, 3), (A, 2, 4) }
Blist = { (B, 1, 5), (B, 2, 7) }
$= 3 \times 5 + 4 \times 7 = 15 + 28 = \underline{\underline{43}}$.

(2,2) Alist = { (A, 1, 3) (A, 2, 4) }
Blist = { (B, 1, 6) (B, 2, 8) }
$= 6 \times 3 + 4 \times 8 = 18 + 32 = \underline{\underline{50}}$